



Igneous Flatstab

Security Assessment

October 31st, 2025 — Prepared by OtterSec

Tamta Topuria

tamta@osec.io

Artem Astakhov

aikasimirov@osec.io

Robert Chen

r@osec.io

Table of Contents

Executive Summary	2
Overview	2
Key Findings	2
Scope	2
Findings	3
Vulnerabilities	4
OS-IFS-ADV-00 Draining Tokens via Negative Fee Arbitrage	5
General Findings	6
OS-IFS-SUG-00 Risk of Admin Lockout	7
OS-IFS-SUG-01 Code Refactoring	8
Appendices	
Vulnerability Rating Scale	9
Procedure	10

01 — Executive Summary

Overview

Igneous labs engaged OtterSec to assess the `flatslab` implementation. This assessment was conducted between September 30th and October 29th, 2025. For more information on our auditing methodology, refer to [Appendix B](#).

Key Findings

We produced 3 findings throughout this audit engagement.

In particular, we identified a vulnerability where negative total fees across swap directions may create risk-free arbitrage loops, allowing users to earn profits indefinitely and drain pool liquidity ([OS-IFS-ADV-00](#)).

we also recommended to require the new admin to be a signer in the instruction that sets the admin, preventing accidental protocol lockout in case of an incorrect new admin address ([OS-IFS-SUG-00](#)). We also made recommendations for updating the codebase to improve functionality and mitigate potential issues by removing unnecessary code and improving error handling ([OS-IFS-SUG-01](#)).

Scope

The source code was delivered to us in a Git repository at <https://github.com/igneous-labs/inf-1.5>. This audit was performed against commit [f34590e](#).

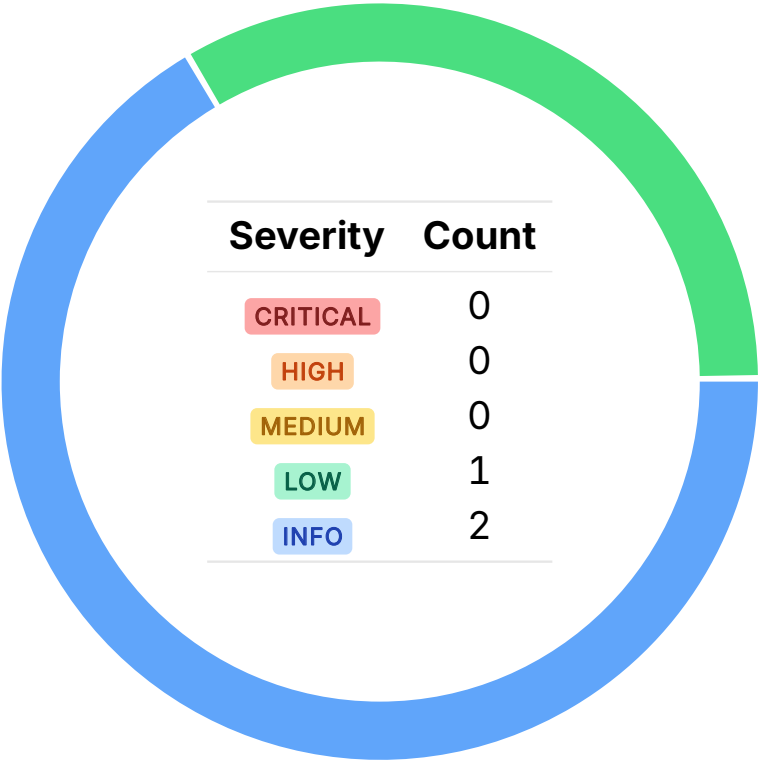
A brief description of the program is as follows:

Name	Description
flatslab	It manages pricing, liquidity, and administrative control for token pairs and LP tokens, ensuring secure mint/redeem operations and consistent fee validation within the protocol.

02 — Findings

Overall, we reported 3 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



03 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in [Appendix A](#).

ID	Severity	Status	Description
OS-IFS-ADV-00	LOW	RESOLVED ✓	Negative total fees across swap directions may create risk-free arbitrage loops, allowing users to earn profits indefinitely and drain pool liquidity.

Draining Tokens via Negative Fee Arbitrage LOW

OS-IFS-ADV-00

Description

This issue arises when a pool allows negative total fees, where the sum of the input and output fees for a swap direction (such as from **A** → **B**) is less than zero. A negative total fee effectively rewards the trader for swapping, resulting in a risk-free arbitrage opportunity. For instance, if the **input_fee** for token *A* and the **output_fee** for token *B* result in a -5% total fee, while the **output_fee** for token *A* and the **input_fee** for token *B* result in a +2% total fee in the same pool (that is, *A* → *B* has a -5% total fee while *B* → *A* has a +2% fee), users may repeatedly swap between *A* and *B* to earn a net 3% profit each cycle of *A* → *B* → *A*.

The risk is amplified in constant-price pools that rely on oracle-derived prices. In such cases, users may repeatedly buy tokens with negative fees until the pool is drained and then sell them in other pools for profit.

Remediation

Ensure total fees for both directions are non-negative: $\text{inpA} + \text{outB} \geq 0$ and $\text{inpB} + \text{outA} \geq 0$.

Patch

Resolved in [PR#88](#).

04 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-IFS-SUG-00	Currently, there is no requirement for the new admin to sign the <code>set_admin</code> instruction, resulting in permanently locking out admin access if an incorrect admin address is provided inadvertently.
OS-IFS-SUG-01	Recommendation for updating the codebase to improve functionality and mitigate potential issues.

Risk of Admin Lockout

OS-IFS-SUG-00

Description

The current `set_admin` instruction only requires the existing admin to sign, allowing a new admin address to be set without verifying its ownership. If there is a mismatch in the provided address, the protocol may become permanently inaccessible.

Remediation

Require the new admin to also be a signer in the `set_admin` instruction to prevent accidental lockout of the protocol.

Patch

Igneous Labs acknowledged the issue and decided to perform a full program redeploy if an incorrect admin is set.

Code Refactoring

OS-IFS-SUG-01

Description

`utils::verify_pks_slice` checks that a list of provided account public keys (`handles`) exactly matches a list of expected public keys (`expected`). However, this function utilizes `zip`, which will silently stop iteration at the shorter of the two arrays, implying that if there is a mismatch in length between `handles` and `expected`, some elements will never be checked.

```
>_ program/src/utls.rs
```

RUST

```
/// [verify_pks`] delegates to this to minimize monomorphization
fn verify_pks_slice<'a, 'acc>(
    abr: &Abr,
    handles: &'a [AccountHandle<'acc>],
    expected: &'a [&[u8; 32]],
) -> Result<(), (&'a AccountHandle<'acc>, &'a [u8; 32])> {
    handles.iter().zip(expected).try_for_each(|(h, e)| {
        if abr.get(*h).key() == *e {
            Ok(())
        } else {
            Err((h, *e))
        }
    })
}
```

Remediation

Utilize `zip_eq` instead of `zip`, which will panic immediately if the arrays are of unequal lengths.

Patch

Igneous Labs acknowledged the issue, noting that the size check is performed before calling the `verify_pks_slice` function and will remain so in future uses of this function.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the [General Findings](#).

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
 - Improperly designed economic incentives leading to loss of funds.
-

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
 - Exploitation involving high capital requirement with respect to payout.
-

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
 - Forced exceptions in the normal user flow.
-

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.
-

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
 - Improved input validation.
-

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.